

Ensemble Learning

Diyah Utami Kusumaning Putri, S.Kom., M.Sc., M.Cs.

Afiahayati, S.Kom., M.Cs., Ph.D.

Department of Computer Science and Electronics

Universitas Gadjah Mada

diyah.utami.k@ugm.ac.id

afia@ugm.ac.id

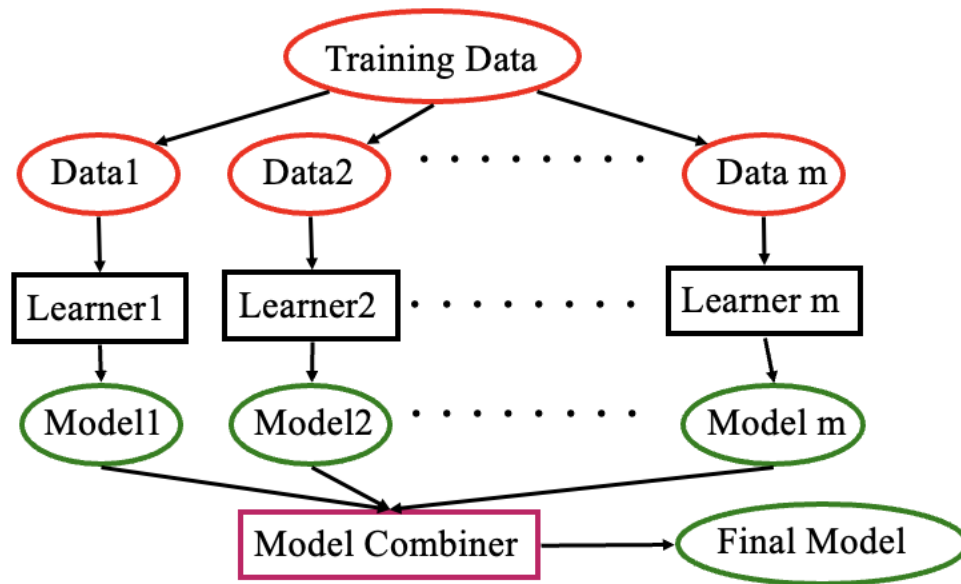


Ensemble Learning

- So far – learning methods that learn a **single hypothesis**, chosen from a hypothesis space that is used to make predictions.
- **Ensemble learning** → select a **collection (ensemble) of hypotheses** and **combine their predictions**.
- **Ensemble learning** is a general meta approach to machine learning that seeks better predictive performance by combining the predictions from multiple models.
- For example, generate 100 different decision trees from the same or different training set and have them **vote on the best classification** for a new example.
- **Key motivation**: reduce the error rate. Hope is that it will become much more unlikely that the ensemble will misclassify an example.

Learning Ensembles

- **Learn multiple alternative definitions of a concept** using different training data or different learning algorithms.
- **Combine decisions** of multiple definitions, e.g. using weighted voting.



Value of Ensembles

- “No Free Lunch” Theorem
→ No single algorithm wins all the time!
- When combining multiple **independent** and **diverse** decisions each of which is at least **more accurate** than random guessing, random errors cancel each other out, **correct decisions are reinforced**.

Example: Weather Forecast

Reality							
1 ^{4/2}							
2							
3							
4							
5							
Combine							

↓ ↓ ↓ ↓ ↓

100%

Intuitions

Majority vote

Suppose we have 5 completely independent classifiers.

- If accuracy is 70% for each
 - $(0.7^5) + 5(0.7^4)(0.3) + 10(0.7^3)(0.3^2)$
 - **83.7% majority vote accuracy**
- 101 such classifiers
 - **99.9% majority vote accuracy**

Note: Binomial Distribution: The probability of observing x heads in a sample of n independent coin tosses, where in each toss the probability of heads is p , is

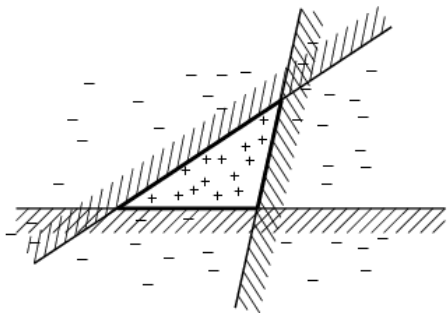
$$P(X = x|p, n) = \frac{n!}{x!(n-x)!} p^x (1-p)^{n-x}$$

Ensemble Learning



Another way of thinking about ensemble learning:

→ Way of **enlarging the hypothesis space**, i.e., the ensemble itself is a hypothesis and the new hypothesis space is the set of all possible ensembles constructible from hypotheses of the original space.

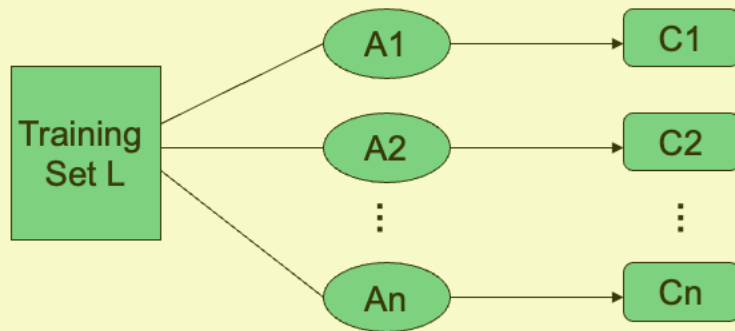


Increasing power of ensemble learning:

- Three linear threshold hypothesis (positive examples on the non-shaded side);
- Ensemble classifies as positive any example classified positively by all three.
- The resulting triangular region hypothesis is not expressible in the original hypothesis space.

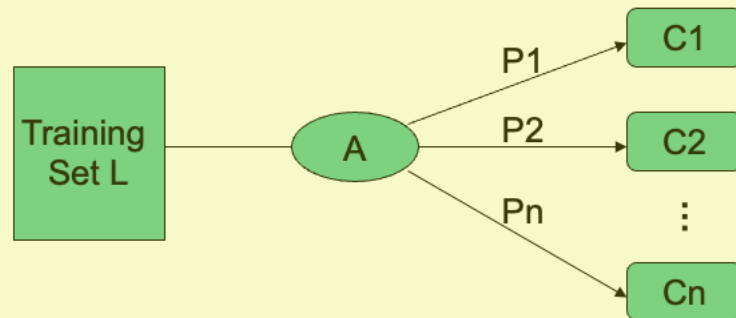
Different Types of Ensemble Learning

(1) Different learning algorithms,
same set of training data



A: algorithm
C: classifier

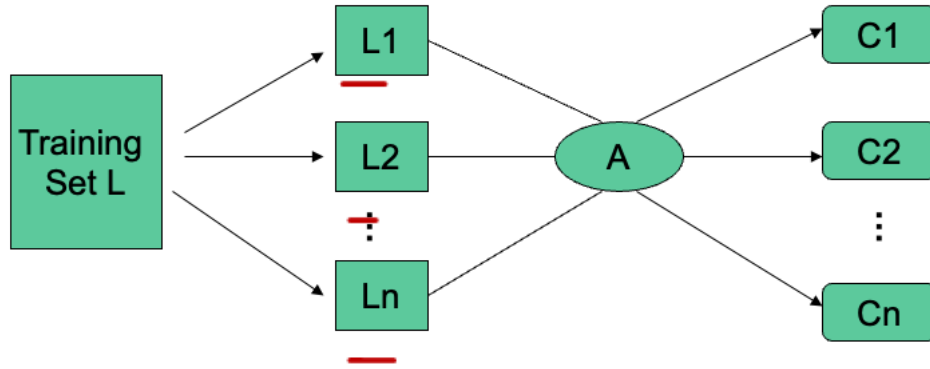
(2) Same algorithm, different
parameter settings



P: parameters for the
learning algorithm

Different Types of Ensemble Learning

- (3) Same algorithm, different versions of data set, e.g.
- Bagging: resample training data
 - Boosting: reweight training data
 - RandomSubSpace (random forest): random subsets of features
 - Decorate: add additional artificial training data



In WEKA, these are called ***meta-learners***, they take a learning algorithm as an argument (***base learner***) and create a new learning algorithm.

Bagging

- Bagging create ensembles by **bootstrap aggregation**, i.e., repeatedly randomly resampling the training data (Leo Breiman, 1996)
- Bagging works by **aggregating** a number of r models that are trained using a number of r datasets that are generated randomly using the **bootstrap method** so that each model is independent of each other.
- **Bootstrap**: draw N items from X with replacement
- The **bootstrap method** (random with replacement) is the generation of training data subsets using *random sampling* with a *uniform distribution*, where data that has been randomly selected can be returned (*replacement*) to the source data so that it can be re-selected.
- **Uniform distribution** is a random distribution with the same probability of being selected for each data object. If the source dataset has N data objects, then each object has probability $P = \frac{1}{N}$ of being selected by random sampling, then the probability of a data object not being selected $Q = \left(1 - \frac{1}{N}\right)^N$. Thus, for a very large N the probability a sample to be selected in the bootstrap process is 63.21%
- Each **bootstrap sample** will on average contain 63.21% of the unique training examples, the rest are replicates.

Bagging Algorithm

Bagging

- Train M learners on M bootstrap samples
- Combine outputs by voting (e.g., majority vote)

Input:

A source dataset S of size m : $S = ((x_1, y_1), (x_2, y_2), \dots, (x_m, y_m))$

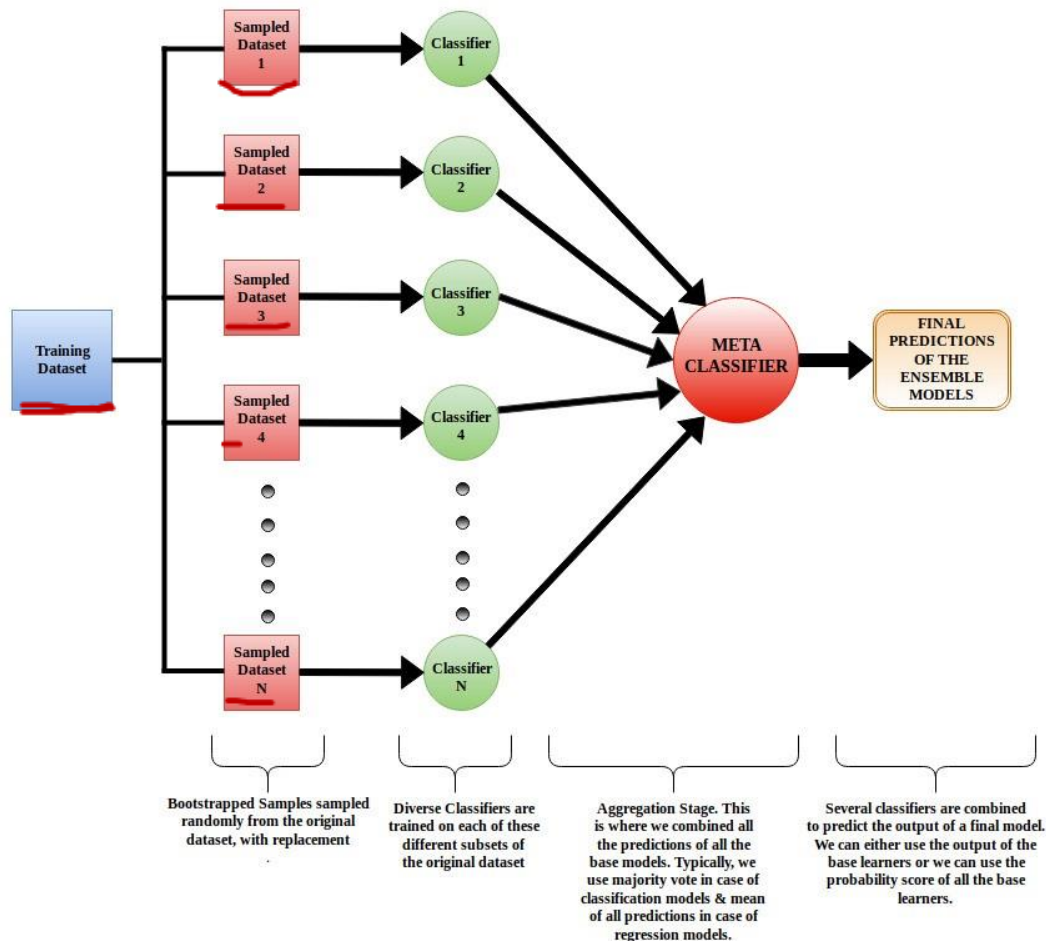
for $t = 1$ until T do

- build a data subset of S using bootstrap with uniform distribution
- build the m_i model using the bootstrap data subset

Output:

Combination result classification model

$$\forall x, F(x) = \text{sign}(\sum_{i=1}^M f_i(x))$$



Bagging Example

A source data set containing 10 data objects with one feature x and class label y

Source Dataset	x	2,4	3,1	3,7	4,4	5,2	5,8	6,4	7,5	8,6	9,4
	y	1	1	1	-1	-1	-1	-1	-1	1	1

Five new data subsets were obtained from five bootstrap processes from the source dataset containing 10 data objects with one feature x and class label y

<u>Bootstrap 1</u>	x	2,4	3,1	3,7	5,2	5,2	6,4	6,4	7,5	8,6	9,4
	y	1	1	-1	-1	-1	-1	-1	-1	1	1
<u>Bootstrap 2</u>	x	2,4	2,4	2,4	3,1	3,7	4,4	4,4	5,2	5,8	8,6
	y	1	1	1	1	-1	-1	-1	-1	-1	1
Bootstrap 3	x	2,4	4,4	4,4	5,2	5,2	5,8	6,4	7,5	8,6	9,4
	y	1	-1	-1	-1	-1	-1	-1	-1	1	1
Bootstrap 4	x	2,4	3,1	3,7	3,7	4,4	4,4	4,4	5,2	7,5	8,6
	y	1	1	1	1	-1	-1	-1	-1	-1	1
<u>Bootstrap 5</u>	x	2,4	3,1	3,7	5,8	7,5	8,6	8,6	8,6	9,4	9,4
	y	1	1	1	-1	-1	1	1	1	1	1

Bagging Example

Five classification models based on C4.5 decision tree, each of which was built using a subset of the results of the bootstrap process

Bootstrap 1	x	2,4	3,1	3,7	5,2	5,2	6,4	6,4	7,5	8,6	9,4
	y	-1	-1	-1	-1	-1	-1	-1	-1	1	1

Model 1: if $x < 8,0$ then
 $y = -1$ else $y = +1$

		Model 1: if $x < 4,0$ then $y = +1$ else $y = -1$									
Bootstrap 2	x	2,4	2,4	2,4	3,1	3,7	4,4	4,4	5,2	5,8	8,6
	y	1	1	1	1	1	-1	-1	-1	-1	-1

[illegible]

Bootstrap 4	x	2,4	3,1	3,7	3,7	4,4	4,4	4,4	5,2	7,5	8,6	Model 4: if $x < 4,0$ then $y = +1$ else $y = -1$
	y	1	1	1	1	-1	-1	-1	-1	-1	-1	

Bootstrap 5	x	2,4	3,1	3,7	5,8	7,5	8,6	8,6	8,6	9,4	9,4
	y	1	1	1	1	1	1	1	1	1	1

Model 5: if $x < 2,0$ then $y = -1$ else $y = +1$

Bagging Example

The bagging process of the five decision tree-based classification models is able to produce combined predictions (sum or majority voting) with 100% accuracy

Original data object x	2,4	3,1	3,7	4,4	5,2	5,8	6,4	7,5	8,6	9,4
<u>Prediction model 1</u>	-1	-1	-1	-1	-1	-1	-1	-1	1	1
Prediction model 2	1	1	1	-1	-1	-1	-1	-1	-1	-1
Prediction model 3	-1	-1	-1	-1	-1	-1	-1	-1	1	1
Prediction model 4	1	1	1	-1	-1	-1	-1	-1	-1	-1
Prediction model 5	1	1	1	1	1	1	1	1	1	1
<u>Sum</u>	1	1	1	-3	-3	-3	-3	-3	1	1
<u>Sign</u>	1	1	1	-1	-1	-1	-1	-1	1	1
Original label y	1	1	1	-1	-1	-1	-1	-1	1	1



This illustration shows that the bagging method will work if every single model is unstable, but complementary. The combination of the five models using the bagging method is able to significantly reduce the prediction error to 0%, which means it provides 100% accuracy. Thus, it can be said that the bagging method can reduce the variance in each single model.

Bagging

- Bagging decreases error by **decreasing the variance** in the results due to ***unstable learners***, algorithms (like decision trees and neural networks) whose output can change dramatically when the training data is slightly changed.
- What is the bagging model error rate? In general, the bagging model provides a lower error rate than the single model (Breiman, 1996).
- How many single models must be built to combine using bagging? Is ~~the more models the~~ more accurate? This depends on the number of dataset. However, in general 50 independent models are sufficient to produce a combined model that gives relatively low error. Increasing the number of single models up to 100 is no longer able to reduce errors (Breiman, 1996)

Boosting



- Boosting was first proposed by Robert E. Schapire in 1990.
- Originally developed by computational learning theorists to guarantee performance improvements on fitting training data for a **weak learner** that only needs to generate a hypothesis with a training accuracy greater than 0.5.
- The boosting method works by boosting a weak initial classification model, sequentially using bootstrap data object sampling based on **dynamic weighting**. The process of strengthening the classification model is sequentially carried out T times until a classification model is produced which is considered strong enough.
- The resulting T models are then combined using a **majority vote with weighting** according to accuracy, not one man one vote as in bagging. In this boosting method, the model with the highest accuracy gets the largest weight, while the model with the lowest accuracy gets the smallest weight.
- Revised to be a practical algorithm, AdaBoost, for building ensembles that empirically improves generalization performance (Freund & Shapire, 1996).

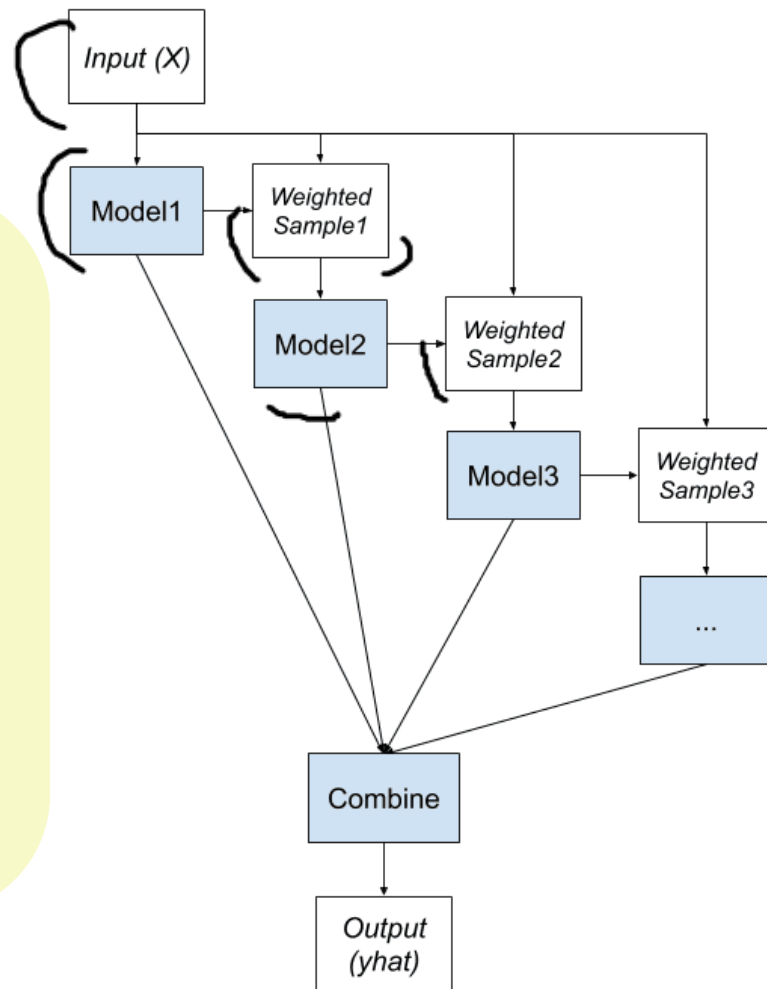
Boosting



Key Insights:

- Instead of sampling (as in bagging) re-weight examples!
- Examples are **given weights**. At each iteration, a new hypothesis is learned (**weak learner**) and the **examples are reweighted** to focus the system on examples that the most recently learned classifier got wrong.
- Final classification based on weighted vote of weak classifiers.

Boosting Ensemble



Construct Weak Classifiers

Using Different Data Distribution

- Start with uniform weighting
- During each step of learning
 - Increase weights of the examples which are not correctly learned by the weak learner
 - Decrease weights of the examples which are correctly learned by the weak learner

Idea

- Focus on difficult examples which are not correctly classified in the previous steps
- Intuitive justification: models should be experts that complement each other

Combine Weak Classifiers

Weighted Voting

- Construct **strong classifier** by **weighted voting of the weak classifiers**

Idea

- Better weak classifier gets a larger weight
- Iteratively add weak classifiers
 - Increase accuracy of the combined classifier through minimization of a cost function

Boosting Algorithm

Input:

A source dataset S of size m : $S = ((x_1, y_1), (x_2, y_2), \dots, (x_m, y_m))$

Initialization:

Maximum iteration T

Weight distribution initialization $\forall i \in \{1, \dots, m\}, D^{(1)}(i) = \frac{1}{m}$

for $t = 1$ until T do

- build a classification model $f_t: \mathcal{R}^d \rightarrow \{-1, +1\}$ using distribution $D^{(t)}$
- calculate $\varepsilon_t = \sum_{i: f_t(x_i) \neq y_i} D^{(t)}(i)$
- select $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$
- update the weight distribution of all data $\forall i \in \{1, \dots, m\}, D^{(t+1)}(i) = \frac{D^{(t)}(i) e^{-\alpha_t y_i f_t(x_i)}}{Z^{(t)}}$ where $Z^{(t)} = \sum_{i=1}^m D^{(t)}(i) e^{-\alpha_t y_i f_t(x_i)}$ is a normalization factor such that $D^{(t+1)}$ leaves a distribution

Output:

Combination result classification model $\forall x, F(x) = \text{sign}(\sum_{t=1}^T \alpha_t f_t(x))$

Boosting Example

First, a bootstrap random sampling is performed on a source data set containing $N=10$ data objects with the same probability $P = \frac{1}{N} = \frac{1}{10} = 0,1$ for each data object. This bootstrap data subset is used to build a model 1 which is still weak and gives a relatively large error.

Source Dataset	x	2,4	3,1	3,7	4,4	5,2	5,8	6,4	7,5	8,6	9,4
	y	1	1	1	-1	-1	-1	-1	-1	1	1
	P	0,1	0,1	0,1	0,1	0,1	0,1	0,1	0,1	0,1	0,1
Bootstrap 1	x	2,4	2,4	3,7	3,7	5,2	5,2	5,8	5,8	7,5	9,4
	y	1	1	1	1	-1	-1	-1	-1	-1	-1

Model 1: if $x < 4,0$ then $y = +1$ else $y = -1$ $\alpha = 1,9459$

Next, data objects that are misclassified by Model 1 have an increased probability of being selected in the next bootstrap, while data objects that are correctly classified have a lower probability of being selected. In **Boosting 2**, the data objects $x=8.6$ and $x=9.4$ which have a high probability of 0.4623 are selected 3 times. With these data, Model 2 makes both data classified correctly.

Source Dataset	x	2,4	3,1	3,7	4,4	5,2	5,8	6,4	7,5	8,6	9,4
	y	1	1	1	-1	-1	-1	-1	-1	1	1
	P	0,0094								0,4623	
Bootstrap 2	x	3,1	4,4	5,8	7,5	8,6	8,6	8,6	9,4	9,4	9,4
	y	-1	-1	-1	-1	1	1	1	1	1	1

Model 2: if $x < 8,0$ then $y = -1$ else $y = +1$ $\alpha = 3,9323$

Boosting Example

Furthermore, the three data objects $x = 2.4$, $x = 3.1$ and $x = 3.7$ which were misclassified by Model 2 were increased their probability to 0.3037. In **Boosting 3**, the three data objects $x = 2.4$, $x = 3.1$ and $x = 3.7$ were selected twice each. With the data from the bootstrap 3, Model 3 is generated which makes the three data objects can be classified correctly. The combination of the three models using the majority vote with weighting according to the accuracy of each single model produces a boosting model with an error of 0%

Source Dataset	x	2,4	3,1	3,7	4,4	5,2	5,8	6,4	7,5	8,6	9,4
	y	1	1	1	-1	-1	-1	-1	-1	1	1
	P	0,3037			0,0009					0,042	

<u>Bootstrap 3</u>	x	2,4	2,4	3,1	3,1	3,7	3,7	5,8	8,6	8,6	9,4
	y	1	1	1	1	1	1	1	1	1	1

Model 3: if $x > 2,0$ then $y = +1$ else $y = -1$ $\alpha = 3,8744$

Combined	x	2,4	3,1	3,7	4,4	5,2	5,8	6,4	7,5	8,6	9,4
	y	1	1	1	-1	-1	-1	-1	-1	1	1

Combined Model: if $x < 4,0$ and $x > 8,0$ then $y = -1$ else $y = +1$

Boosting

- How many models should be built? Depends on the data set you have. In general, however, 50 independent models will suffice.
- In (Breiman 1996a) it is stated that until more use, up to 100, the independent model no longer improves accuracy.
- The boosting performance is relatively high compared to the stand-alone single model (Breiman 1996a). In general, boosting performance is higher than bagging (Quinlan 1996)

The Difference between Bagging and Boosting

Bagging	Boosting
Performs bootstrapping with a uniform distribution (where data objects have an equal chance of being selected) in parallel so that each data subset is independent. Each subset of data is used to build an independent model.	Performs sequential bootstrapping. Bootstrap was first performed with a uniform distribution (each data has the same weight or probability to be selected) and the results are used to train models that are still weak (weak). The prediction results of this weak model are used to update the weights of the data objects, where the weights of the incorrectly predicted data objects are increased, while, the weights that are correctly predicted are decreased. The second bootstrap is then carried out based on the new weights and the results are used to train the second model (which is expected to be stronger than the first model). And so on until the resulting model gets stronger and stronger.
The final decision is obtained by majority voting without weight, which means one man one vote .	The final decision on boosting is made with a certain weighting scheme , where models with high accuracy are given a large weight, while models with low accuracy are given a small weight, not one man one vote.

Experimental Results on Ensembles

(Freund & Schapire, 1996; Quinlan, 1996)

- Ensembles have been used to improve generalization accuracy on a wide variety of problems.
- On average, Boosting provides a larger increase in accuracy than Bagging.
- Boosting on rare occasions can degrade accuracy.
- Bagging more consistently provides a modest improvement.
- Boosting is particularly subject to over-fitting when there is significant noise in the training data.
- Bagging is easily parallelized, Boosting is not.

Random Forest



- Random forest or random ensembles is one variant of bagging. (Reference: Leo Breiman, *Random Forests*, Machine Learning, 45, 5-32, 2001)
- Random forest is a combination of decision trees such that each tree depends on the values of a random vector that is sampled independently and with the same distribution for all trees in the forest.
- The strength of random forest lies in the random selection of features to sort each node which is capable of producing a relatively low error rate.
- The difference between RF and bagging: Bagging uses all the attributes to build an independent model whereas RF only uses some of the features (usually 20% of the total features). In this way, RF will be more computationally efficient. The set of independent models built by RF is also more varied than bagging.

Random Forest



- **Motivation:** reduce error correlation between classifiers
- **Main idea:** build a larger number of un-pruned decision trees
- **Key:** using a random selection of features to split on at each node

How Random Forest Work

- Each tree is grown on a bootstrap sample of the training set of **N** cases.
- A number **m** is specified much smaller than the total number of variables **M** (e.g. $m = \sqrt{M}$).
- At each node, **m** variables are selected at random out of the **M**.
- The split used is the best split on these **m** variables.
- Final classification is done by majority vote across trees.

Advantages of Random Forest

- Error rates compare favorably to Adaboost
- More robust with respect to noise.
- More efficient on large data
- Provides an estimation of the importance of features in determining classification
- More info at: http://stat-www.berkeley.edu/users/breiman/RandomForests/cc_home.htm

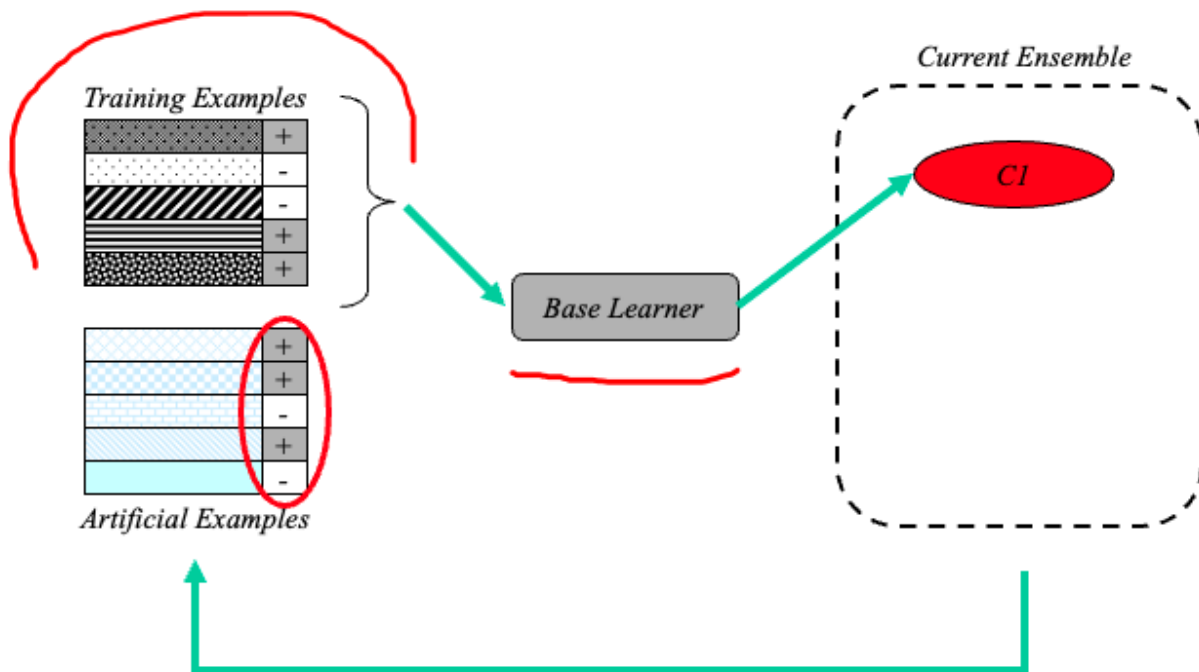
DECORATE

(Melville & Mooney, 2003)

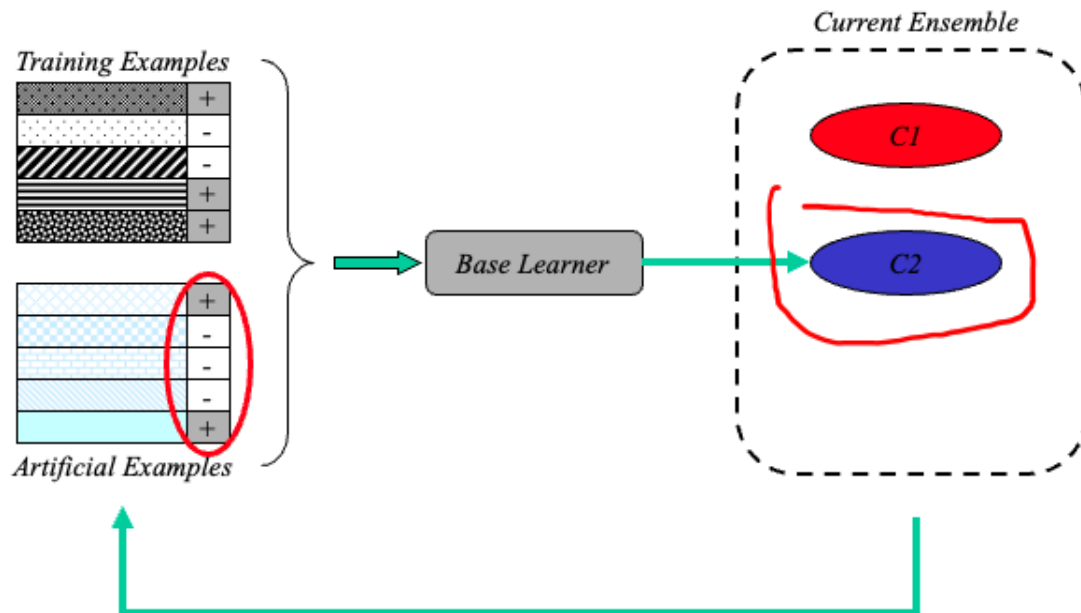


- Change training data by adding new artificial training examples that encourage diversity in the resulting ensemble.
- Improves accuracy when the training set is small, and therefore resampling and reweighting the training set has limited ability to generate diverse alternative hypotheses.

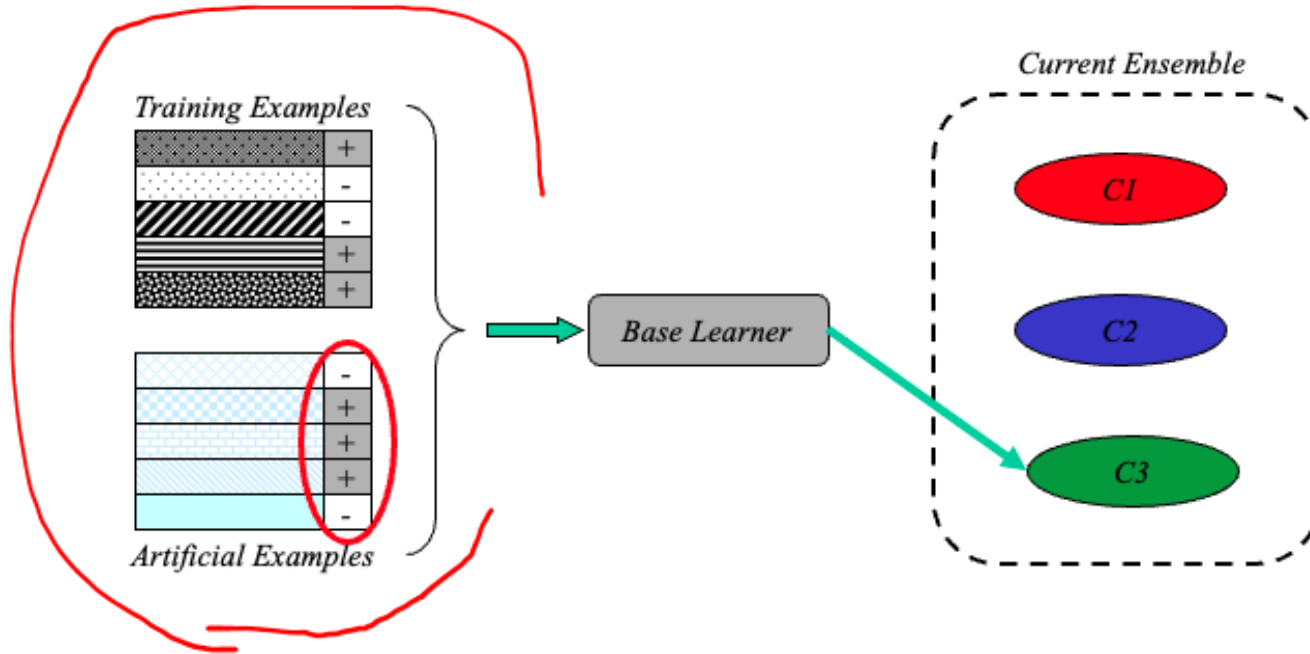
Overview of DECORATE



Overview of DECORATE



Overview of DECORATE

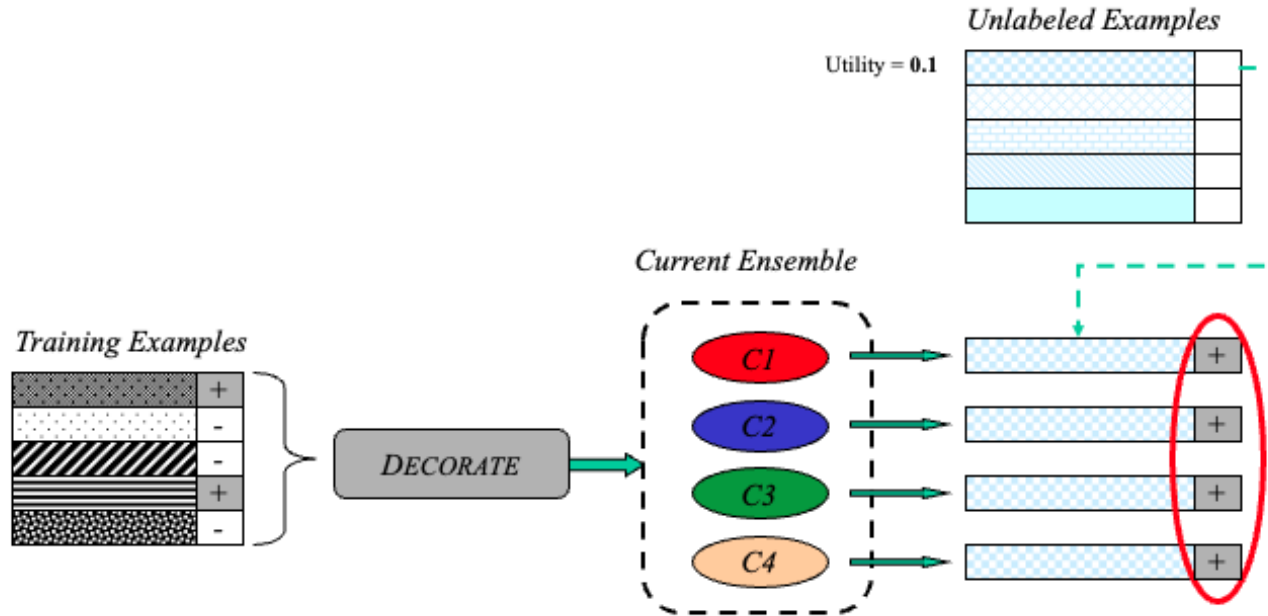


Ensemble and Active Learning

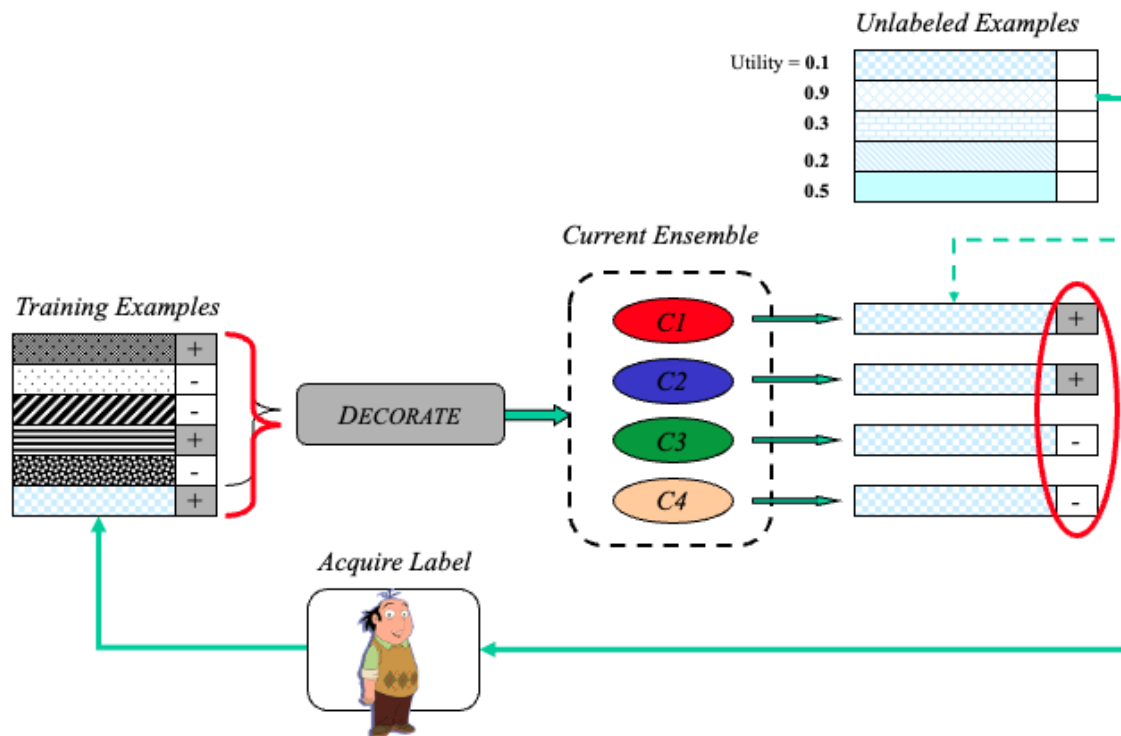


- Ensembles can be used to actively select good new training examples.
- Select the unlabeled example that causes the most disagreement amongst the members of the ensemble.
- Applicable to any ensemble method:
 - QueryByBagging
 - QueryByBoosting
 - ActiveDECORATE

Active-DECORATE



Active-DECORATE



Combining an Ensemble of Classifiers

Voting:

- Classifiers are combined in a static way.
- Each base-level classifier gives a (weighted) vote for its prediction.
- Plurality vote: each base-classifier predict a class
- Class distribution vote: each predict a probability distribution

$$p_C(x) = \sum_{C \in \mathcal{C}} p_C(x) / |\mathcal{C}|$$

Stacking: a stack of classifiers

- Classifiers are combined in a dynamically.
- A machine learning method is used to learn how to combine the prediction of the base-level classifiers.
- Top level classifier is used to obtain the final prediction from the predictions of the base-level classifiers

Cascading:

- Combine classifiers iteratively.
- At each iteration, training data set is extended with the prediction obtained in the previous iteration

Voting

- **Voting** is one of the simplest ways of combining the predictions from multiple machine learning algorithms.
- Building multiple models (typically of differing types) and simple statistics (like calculating the mean) are used to combine predictions.
- It works by first creating two or more standalone models from your training dataset. A Voting Classifier can then be used to wrap your models and average the predictions of the sub-models when asked to make predictions for new data.
- You can create a voting ensemble model for classification using the *Voting Classifier* class in sklearn.

Stacking



- The stacking technique was first proposed by David H. Wolpert in 1992 in his paper entitled Stacked Generalization (Wolpert 1992).
- **Stacked generalization** is defined as a scheme to minimize the generalization error rate of one or more generalizer models.
- **Stacked generalization works** by inferring the biases of one or more generalizing models to the training data set, where generalization is carried out in the second stage with input in the form of one or more predictions of the original generalizer model.
- **Generalizer** is a classification or regression model that is built using learning techniques with a certain training data set so that it can be used to generalize new data that has never been trained.

Stacking



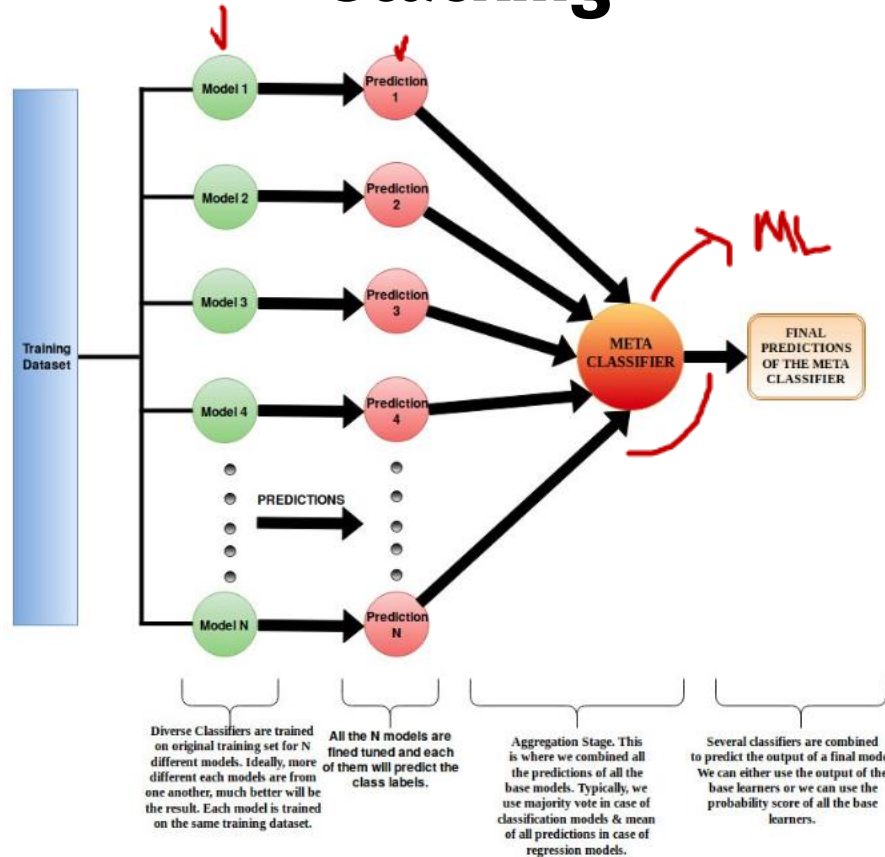
- Stacked generalizations can be constructed using one or more generalizer models.
- If built using only **a single generalizer model**, stacked generalization can be viewed as a scheme for estimating (and subsequently correcting) the error of the generalizing model that has been trained using a certain training data set and then used to generalize new data that has never been trained.
- If built using **more than one single generalizer model**, stacked generalization can be viewed as a very complex version of cross-validation, which uses a more complicated strategy than the winner-takes-all scheme of the original cross-validation to combine a number of these single generalization models.

Stacking



- **Stacking** is an ensemble learning technique which is used to combine the predictions of diverse classification models into one single model also known as the meta-classifier.
- All the individual models are trained separately on the complete training data set and fine-tuned to achieve greater accuracy. The bias and variance trade-off is taken care off for each model.
- In stacking, a new model is trained based on the predictions made by the previous models.
- This process takes place **sequentially**. This process occurs multiple times based on how many layers of stacking one would like to use.
- The final stage consists of one single powerful model, which gives us the final output by combining the output of all the models present in the previous layers.
- This single powerful model at the end of a stacking pipeline is called the meta-classifier.
- Often times, using stacking classifiers increases the prediction accuracy of a model.
- But in no way can there be a guarantee that using stacking will increase the prediction accuracy at all times!

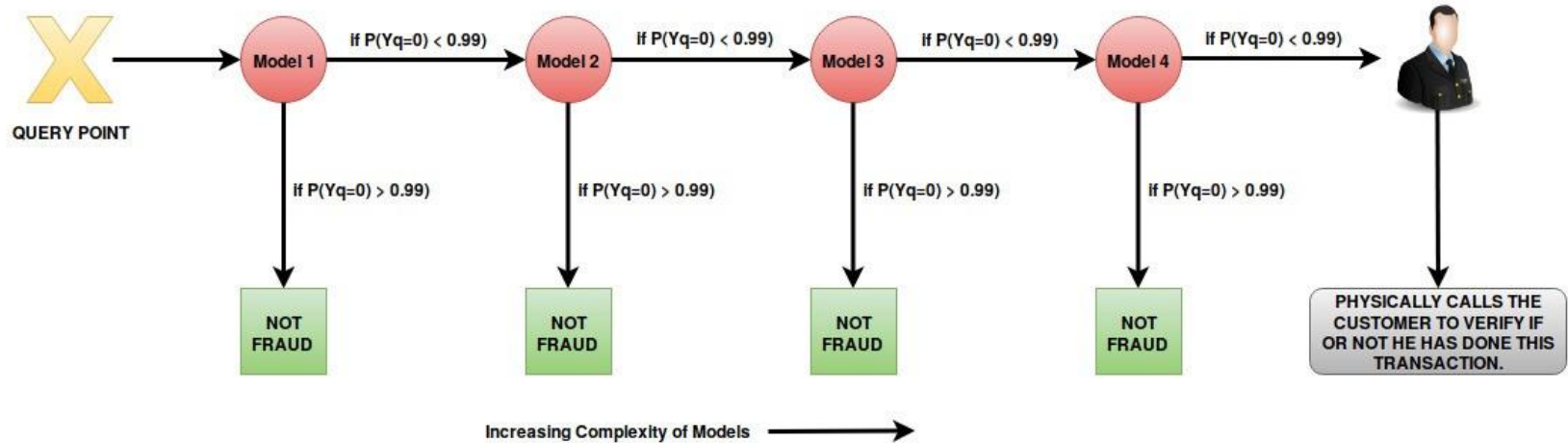
Stacking



Cascading

- **Cascading** is one of the most powerful ensemble learning algorithm which is used by ML engineers and scientists when they want to be absolutely dead sure about the accuracy of a result.
- Cascading is mostly used in scenarios where you cannot afford to make a mistake. This class of models are very very accurate.
- For example, a cascading technique is mostly used to detect fraudulent credit card transactions, or maybe when you want to be absolutely sure that you don't have cancer.

Cascading Example



DIFFERENT STAGES OF QUERING CASCADE CLASSIFIERS IN A FOUR MODEL CASCADE SYSTEM

- If you think about it, it's a binary classification problem where a class label 0 means the transaction is not fraud & a class label 1 means the transaction is fraudulent.
- In such a model, it's very risky to put our faith completely in just one model.
- So what we do is build a sequence of models (or a cascade of models) to be absolutely sure about the fact that the transaction is not fraudulent.

Summary

There are three important things to consider when choosing an ensemble learning:

- Number of data objects: hundreds, thousands, millions or more
- Number of features: tens, hundreds, thousands or more
- Number of classes: two (binary class) or more (multi class)